

智能法务系统 —— 智能体架构设计文档

版本信息

版本	日期	说明
v1.0	2026-06-14	初稿

一、设计动机

1.1 为什么需要 Agent

传统的"一次 LLM 调用"模式在法务场景中面临三个天花板：

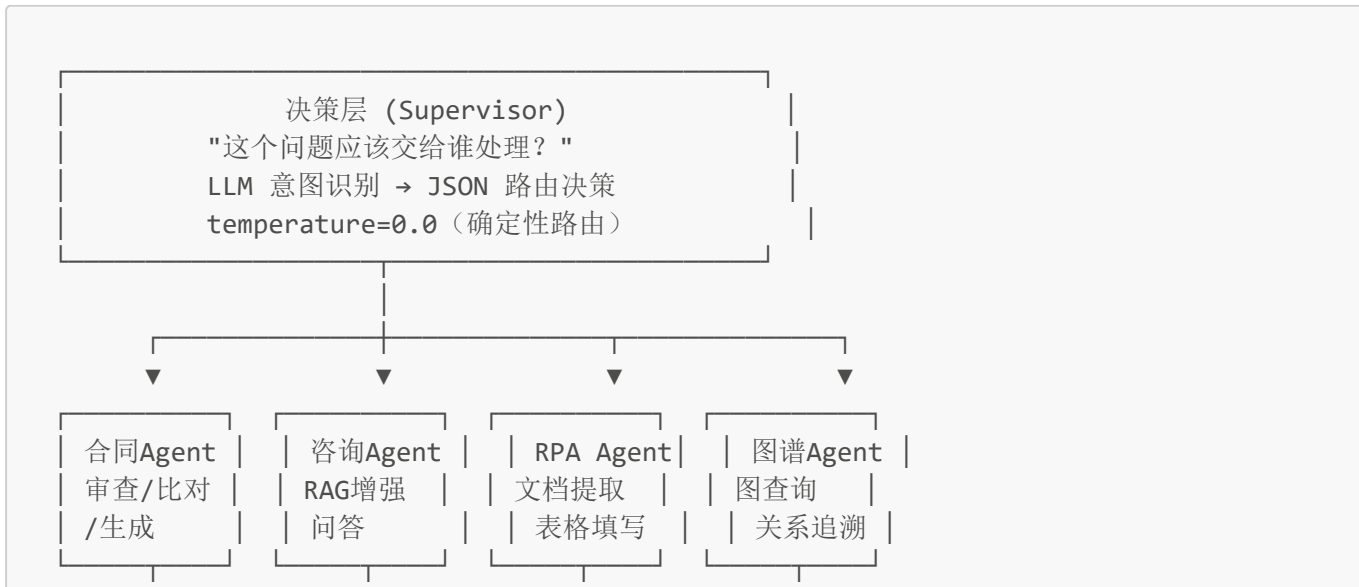
- 任务多样性**：合同审查、法律咨询、文档提取、图谱查询四类任务差异显著，单一 Prompt 模板无法有效覆盖
- 工具调用**：法律咨询需要先检索法条和判例再生成回答，这不仅是 LLM 的生成能力问题，更是信息获取能力问题
- 多步推理**：复杂的法务任务可能需要多个处理步骤——先审查合同、发现高风险条款、再检索相关判例、最后生成法律意见——单次调用无法完成

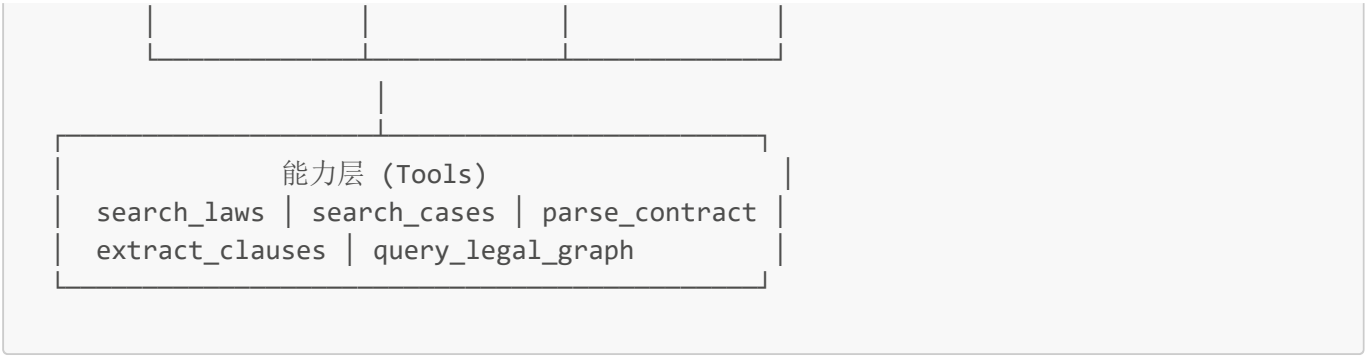
Agent 的本质是将 LLM 从"回答问题的人"升级为"调度任务的指挥官"：指挥官不需要亲自审合同、查法条、写报告，他只需要判断"这个任务应该分给谁"，然后分下去，由专业 Agent 执行。

1.2 设计原则

- 关注点分离**：意图识别与任务执行分离——Supervisor 只做路由，Agent 只做执行
- 弹性架构**：新增 Agent 或工具不需要改动现有节点，只需要注册到编排器
- 渐进可用**：从单 Agent 到多 Agent 协作的演进路径清晰，不需要架构重构
- 容灾优先**：任何 LLM 调用都有备用路径，确保用户请求不丢失

二、整体架构：三层 Supervisor 模型





三层解耦后，增加一个新 Agent 只需要在 Supervisor 的 Prompt 中加一行规则，在编排器中注册一个新节点和边。增加一个新工具只需要写一个 `@tool` 函数并加入工具注册表。

三、技术选型：LangGraph Supervisor

系统选用 LangGraph 作为 Agent 编排框架，基于以下考量：

- 1. **有向状态图**：LangGraph 以 StateGraph 建模 Agent 协作，支持条件分支和循环，比简单的链式调用更适合需要路由的场景
- 2. **状态管理**：AgentState 在节点间自动传递，框架负责状态持久化和回滚
- 3. **条件路由**：`add_conditional_edges` 机制使 Supervisor 节点可以根据运行时决策动态路由到不同 Agent
- 4. **Python 原生**：与 FastAPI 后端技术栈一致，无需引入新的语言或运行时

四、状态设计：AgentState

AgentState 是所有节点共享的上下文容器，定义为 TypedDict，包含 7 个字段：

字段	类型	写入者	读取者	语义
user_input	str	入口	Supervisor	用户原始输入文本
task_type	str	Supervisor	各 Agent	任务类型枚举（contract_review / consultation / rpa / kg_query）
next_agent	str	Supervisor	_route()	路由目标 Agent 名称
messages	list	LangGraph 自动	各节点	多轮对话历史
context	dict	各节点	各节点	节点间传递的中间结果（扩展槽）
result	str	执行 Agent	出口	最终返回给用户的文本
finished	bool	执行 Agent	图引擎	是否完成（True → END）

设计要点：

- 用 TypedDict 而非 Pydantic BaseModel，确保 JSON 序列化兼容
- context 字段是扩展槽——未来多 Agent 串联时传递中间结果（如 contract_agent 的审查结果传递给 kg_agent 作为检索上下文）

- finished 是图的终止条件：任何 Agent 设为 True 后图执行结束

五、图结构设计

5.1 节点注册

编排器注册 5 个节点：1 个 Supervisor 加 4 个专业 Agent。

```
workflow = StateGraph(AgentState)
workflow.add_node("supervisor", self._supervisor_node)
workflow.add_node("contract_agent", self._contract_node)
workflow.add_node("consultation_agent", self._consultation_node)
workflow.add_node("rpa_agent", self._rpa_node)
workflow.add_node("kg_agent", self._kg_node)
```

5.2 入口与路由

Supervisor 作为唯一入口。执行后通过条件路由分发到对应 Agent：

```
workflow.set_entry_point("supervisor")
workflow.add_conditional_edges("supervisor", self._route, {
    "contract_agent": "contract_agent",
    "consultation_agent": "consultation_agent",
    "rpa_agent": "rpa_agent",
    "kg_agent": "kg_agent",
    "end": END, # 无输入或异常时直接结束
})
```

5.3 Agent 终止

当前为单次执行模式——各 Agent 完成后直接结束：

```
workflow.add_edge("contract_agent", END)
workflow.add_edge("consultation_agent", END)
workflow.add_edge("rpa_agent", END)
workflow.add_edge("kg_agent", END)
```

演进路径：将 END 改回 "supervisor" 即可形成循环——Agent A 完成后 Supervisor 重新评估是否需要 Agent B 继续处理。

5.4 条件路由函数

```
@staticmethod
def _route(state: AgentState) -> str:
```

```
return state.get("next_agent", "end")
```

设计极其简单——只读 next_agent 字段，不包含任何业务判断。Supervisor 写什么，它就路由到哪里。

六、Supervisor 节点设计

6.1 Prompt 工程

Supervisor 的系统提示词定义了四个 Agent 的能力边界：

- contract_agent：合同审查、条款比对、合同生成、风险识别
- consultation_agent：法律问题解答、法律知识检索、案例查询
- rpa_agent：文档数据提取、表格填写、自动化操作
- kg_agent：知识图谱查询、案例关系追溯

要求 LLM 输出 JSON 格式：{"task_type": "...", "agent": "...", "reason": "..."}

6.2 关键参数

temperature 设为 0.0——Supervisor 是确定性组件，同样的输入必须产生同样的路由结果。不需要创造力，需要稳定性和一致性。

6.3 路由规则

用户意图关键词	路由目标	task_type
合同 / 审查 / 条款 / 比对 / 生成	contract_agent	contract_review
咨询 / 法律 / 问答 / 检索	consultation_agent	consultation
提取 / 表格 / RPA / 自动化	rpa_agent	rpa
图谱 / 关系 / Neo4j / 追溯	kg_agent	kg_query

6.4 容灾机制

当 LLM 返回的 JSON 解析失败（格式异常、缺少引号、多余换行），降级为关键词匹配：

1. 检测"合同 / 审查 / 条款 / 比对" → contract_agent
2. 检测"图谱 / 知识图谱 / 案例关系" → kg_agent
3. 其余所有情况 → consultation_agent（默认兜底）

设计原则：宁可路由不够精准（错误的 Agent 可以引导用户重新描述），不能让用户请求丢失（直接报错）。

七、六个 Agent 的职责与能力

系统实际存在 6 个 Agent，其中 4 个注册在编排器中，2 个通过直接路径调用。

7.1 编排器注册的 4 个 Agent

Agent	节点状态	实际业务类	文件
contract_agent	占位（返回确认文本）	ContractAgent	contract_service/agent.py
consultation_agent	占位	ConsultationAgent	consultation_service/agent.py
rpa_agent	占位	RPAAgent	agent_service/rpa_agent.py
kg_agent	占位	KGQuery	knowledge_graph/query.py

7.2 不在编排器中的 2 个 Agent

Agent	文件	功能	调用路径
CaseAnalyzer	case_analysis_service/analyzer.py	案情要素提取、判例匹配、分析报告生成	case_routes.py 直接调用
KnowledgeExtractor	knowledge_graph/extractor.py	法律文本实体关系抽取	GraphBuilder 调用

7.3 各 Agent 功能详述

ContractAgent——输入：合同全文及条款列表；输出：每个条款的风险等级（high/medium/low/none）、法律依据、修改建议。调用 LLM 1-3 次（审查 1 次、比对 1 次、生成 1 次）。调用审计日志。代码在 `contract_service/agent.py`。

ConsultationAgent——输入：自然语言问题；输出：结构化法律意见（直接回答→法律依据→实操建议）及引用法条列表。流程：先通过 HybridRetriever 执行混合检索（ChromaDB语义+Whoosh关键词），检索结果作为 LLM Prompt 的上下文注入，LLM 基于真实法条生成回答。调用 LLM 1 次。代码在 `consultation_service/agent.py`。

RPAAgent——输入：DOCX/PDF 合同文件；输出：结构化 JSON（合同标题、甲乙双方、签署日期、金额、关键条款、争议解决方式）。支持 python-docx 读取 DOCX，pdfplumber/PyPDF2 读取 PDF，LLM 提取字段。调用 LLM 1 次。代码在 `agent_service/rpa_agent.py`。

KGQuery——输入：关键词或 Cypher 查询语句；输出：匹配的法律实体列表及关联实体网络。通过 Neo4j 全文索引搜索。不调用 LLM（纯图查询）。代码在 `knowledge_graph/query.py`。

CaseAnalyzer——输入：案件类型及结构化事实；输出：案情摘要、法律适用分析、相似判例匹配、证据清单、费用估算、诉讼时效检查。调用 LLM 2 次（要素提取+判例匹配）。代码在 `case_analysis_service/analyzer.py`。

KnowledgeExtractor——输入：非结构化法律文本；输出：识别出的法律实体（法规/法条/判例/法院/概念）及关系列表。调用 LLM 1 次。代码在 `knowledge_graph/extractor.py`。

八、工具层设计

工具层提供 6 个 `@tool` 函数，供 Agent 在执行过程中按需调用：

工具	功能	调用目标
search_laws(query)	检索法律法规	hybrid_retriever.search_laws()

工具	功能	调用目标
search_cases(query)	检索判例	hybrid_retriever.search_cases()
search_contracts(query)	检索相似合同条款	hybrid_retriever.search()
parse_contract(path)	解析合同文档提取条款	legal_parser.parse_to_documents()
extract_clauses(text)	从文本提取结构化条款	legal_parser
query_legal_graph(cypher)	查询知识图谱	kg_query.search_fulltext()

工具与 Agent 分离——"怎么查"属于工具层，"查完之后怎么用"属于 Agent 层。检索策略升级（如从混合检索升级为重排序模型）只改工具函数，六个 Agent 一行不变。

九、两条调用路径

系统提供两条路径到达 LLM，服务于不同场景：

路径 A：直接路径（当前生产路径）

```
用户 → API 路由 → 对应 Agent 类 → LLMClient
```

优点：链路短、延迟低。适用于用户明确知道要做什么的场景（点击"合同审查"按钮）。

路径 B：编排路径（架构预备路径）

```
用户 → AgentOrchestrator.run() → Supervisor → 专业 Agent → LLMClient
```

优点：统一入口、支持多 Agent 协作。适用于用户在一个对话中提出复合需求的场景（"先审查合同，再查相关判例，再生成法律意见书"）。

当前编排器的 4 个 Agent 节点为占位实现——Supervisor 路由通了、状态流转验证了、容灾回退生效了，但节点内部只返回确认文本。真实业务逻辑通过路径A直接调用 Agent 类完成。两条路径的并存不是冗余——它们分别服务于当前的单步任务和未来的多步协作。

十、审计集成

所有 Agent 调用 LLM 时，自动通过 AuditLogger 记录一条审计日志，包含：操作时间、用户标识、任务类型、使用模型、输入输出摘要、Token 消耗、响应延迟。日志以追加写入模式存储，每条一行 JSON，SHA256 脱敏感字段。

审计日志实现了以下目标：

- 1. 全链路追溯——从业务数据（合同编号/咨询编号）可追踪到对应的 LLM 调用记录
- 2. 不可篡改——追加写入模式使日志天然防删除、防修改
- 3. 合规验证——审计员可通过 audit_id 验证 AI 决策过程的完整性和合规性

十一、架构弹性与演进路径

当前 Agent 架构为三个方向的扩展预留了清晰的接入点，每一步都不需要重构：

1. **加 Agent**：两步——在 Supervisor Prompt 加一行规则 + 在编排器 add_node + add_edge
2. **加工具**：两步——写 @tool 函数 + 注册到 ALL_TOOLS 列表
3. **多 Agent 串联**：一步——把 add_edge("xxx_agent", END) 的 END 改成 "supervisor"

架构遵循"先搭框架、后填内容"的渐进式策略：Supervisor 路由和状态流转本身具备独立可验证性（能通过占位节点确认"路由到了正确的 Agent"），这降低了后续填入真实业务逻辑时的调试复杂度。